Midterm Study Guide

Sessions 1–7

Exam date: Friday 2026-05-22
Format: multiple-choice ("tipo test")
Goal: 100 / 100

How to use this guide

- Each section condenses one session into the most exam-likely content.
- **Bold = high-value memorization.** *Italic asides flag gotchas.*
- After reading: take the 50-question practice quiz (separate file).

Recommended pace: 1 session per day for 7 days, then 3-day practice, then 2-day review.

Session 1 — Intro to Analytics Engineering

Analytics Engineering = applying **software engineering principles** (version control, CI/CD, automated testing, modular code) to data transformation. It bridges Data Engineering (ingestion) and Data Analysis (reporting).

ADLC — Analytics Development Lifecycle, 4 stages:
Build → Collaborate → Deploy → Monitor

ELT vs ETL:

	ETL	ELT
Transform happens	**Before** loading	**After** loading (in the warehouse)
Compute	Staging server	Warehouse
Tools	Informatica, SSIS	**dbt**, SQL
Modern stack?	Legacy	Standard

dbt is the "T" in ELT — transforms data already in your warehouse. dbt does NOT extract or load.

Session 1 (cont.) — Roles, dbt, certification

Data team roles (left → right):

- **Data Engineer** — pipelines, ingestion (E + L)
- **Analytics Engineer** — transforms (T) → trusted, documented datasets
- **Data Analyst** — consumes data → insights

What dbt is — *a compiler + runner*:

- **Compiler**: Jinja + SQL → raw SQL
- **Runner**: executes that SQL against the warehouse
- **Control plane**: testing, docs, lineage in one place

dbt Certification = 65 MC questions, 2 hours, 65% pass. **8 domains**:

1. Developing dbt models · 2. Model governance · 3. Debugging modeling errors
2. Managing data pipelines · 5. Implementing dbt tests · 6. Creating/maintaining docs
3. Implementing external dependencies · 8. Leveraging dbt state

Session 2 — Setup & DuckDB

DuckDB = "SQLite for analytics." **In-process** (no server), **columnar**, **reads Parquet/CSV directly**. Our `my_database.duckdb` file IS the warehouse.

Environment — uv (Python package manager). Activation:

- macOS/Linux: `source .venv/bin/activate`
- Windows PS: `.\.venv\Scripts\Activate.ps1`
- Skip activation: prefix with `uv run`

Load data: `python create_db.py` → 14 raw tables.
Verify connection: `dbt debug`

Profile resolution order (first match wins):

1. `--profiles-dir` CLI flag
2. `DBT_PROFILES_DIR` env var
3. **Project directory** (next to `dbt_project.yml`) ← our setup
4. `~/.dbt/profiles.yml` global

Session 2 (cont.) — Project structure

```
dbt-ie/
├── dbt_project.yml    # project config (name, paths, defaults)
├── profiles.yml       # connection (adapter, path, schema)
├── packages.yml       # external packages
├── models/            # SQL + Python transformations
│   ├── staging/       │   ├── intermediate/   │   └── marts/
├── seeds/             # static CSVs → tables
├── macros/            # reusable Jinja
├── snapshots/         # SCD Type 2 (later)
├── tests/             # singular custom tests
├── analyses/          # ad-hoc SQL — NEVER run by dbt run
├── target/            # compiled + run artifacts (gitignored!)
└── logs/              # dbt logs (gitignored)
```

Naming conventions: `stg_*` (staging) · `int_*` (intermediate) · `dim_*` (dimensions) · `mart_*` (facts)

Session 2 (cont.) — ref() & target/

`ref()` matters because:

- Resolves schema/table **at compile time** (portable across dev/staging/prod)
- Builds the **DAG** automatically — dbt knows correct build order
- Safe refactoring — rename a model and downstream consumers are flagged

`target/` directory:

- `target/compiled/` — Jinja rendered, no DDL. *What dbt would send.*
- `target/run/` — Wrapped in materialization DDL (`CREATE VIEW AS ...`). *What dbt actually executed.*

Always check compiled SQL when debugging — it is ground truth.

Foundations — DB vs DWH, dbt's role

Database (OLTP) vs Data Warehouse (OLAP):

	DB (OLTP)	DWH (OLAP)
Job	Run the app	Answer analytical questions
Workload	Many small reads/writes	Big scans + aggregations
Rows/query	1s–1000s	Millions
Examples	Postgres, MySQL	Snowflake, BigQuery, **DuckDB**

dbt IS / IS NOT:

dbt IS	dbt is NOT
SQL compiler + runner	A database/warehouse
Dependency + testing framework	An ETL/ingestion tool
Documentation generator	A scheduler (out of the box)
Git-versioned transformation code	A BI/dashboarding tool

Foundations (cont.) — `dbt run` flow

Memorize the order:

1. **Reads** → `dbt_project.yml` + `profiles.yml` + `models/`
2. **Parses** → builds DAG from `ref()` and `source()` calls
3. **Compiles** → Jinja → pure SQL → `target/compiled/`
4. **Connects** → opens DuckDB via `profiles.yml`
5. **Executes** → models in DAG order, materializes tables/views
6. **Logs** → writes `run_results.json` + `manifest.json` to `target/`

`target/` is disposable — delete anytime, regenerate.

Session 3 — Sources, Staging, Seeds

Sources = raw data in the warehouse, loaded by an EL tool (or `create_db.py`).
Defined in `models/sources.yml`. Reference with `{{ source('raw', 'customers') }}`.

Staging Do's: rename, cast types, basic cleaning, 1:1 mapping to source.
Staging Don'ts: NO joins, NO aggregations, NO business logic.

Seeds = static CSVs in `seeds/`. Loaded with `dbt seed`. Reference with `{{ ref('segments') }}` (same as a model).
Do NOT use for large raw data.

Source freshness in `sources.yml`:

```yaml
freshness:
  warn_after:  {count: 12, period: hour}
  error_after: {count: 24, period: hour}
loaded_at_field: loaded_at
```

Run: `dbt source freshness`

Session 3 (cont.) — Tests & build

Generic tests (4 built-in):

Test	Checks
`unique`	No duplicate values
`not_null`	No null values
`accepted_values`	Values within a known set
`relationships`	Foreign key integrity

Defined in YAML next to the model:

```yaml
columns:
  - name: customer_id
    tests: [unique, not_null]
```

`dbt build` = `run` + `test` + `seed` + `snapshot` in DAG order. **Prefer over** `dbt run`. If a test fails, downstream models won't run — bad data is contained.

Codegen package generates boilerplate:

- `codegen.generate_source(...)` → sources.yml
- `codegen.generate_base_model(...)` → staging SQL

Session 4 — DRY, layers, DAG

DRY = Don't Repeat Yourself. Build once, `ref()` many times.
Benefits: maintainability, consistency (one definition of "revenue").

The four layers:

1. **Source / Seed** (raw)
2. **Staging** (`stg_*`) — 1:1 with source, **views**, cleaning/renaming/casting only
3. **Intermediate** (`int_*`) — logic-concentric, joins/dedupe/calculated fields, internal use only
4. **Marts** (`dim_*`, `mart_*`) — business-ready, materialized as **tables**

DAG — Directed Acyclic Graph:

- **Directed**: data flows source → mart (one way)
- **Acyclic**: no loops (a model can't depend on itself)
- **Graph**: nodes = models, edges = `ref()` / `source()` calls

dbt builds the DAG automatically — never written by hand.

Session 4 (cont.) — Selectors

Memorize selector syntax (`-s` / `--select`):

Syntax	Meaning
`mart_orders`	just this model
`+mart_orders`	this + **upstream** (ancestors)
`mart_orders+`	this + **downstream** (descendants)
`+mart_orders+`	this + both directions
`1+mart_orders`	this + **1 level** upstream only
`staging`	everything in the staging folder
`tag:finance`	everything tagged "finance"
`path:models/marts`	everything in that path

Session 4 (cont.) — Five SQL patterns

1. **Cleaning & casting** (staging) — `trim(lower(email))`, `::date`, `coalesce`
2. `case when` **classification** — encode business buckets once
3. **Joins** (intermediate) — preserve grain, comment why left vs inner
4. **Aggregations** (marts) — `group by` defines the grain
5. **Window functions** — `row_number() over (partition by ...)` for rank/running total

Python models — when SQL is awkward (text parsing, ML, APIs):

```python
def model(dbt, session):
    dbt.config(materialized="table")
    df = dbt.ref("dim_customers")
    return df.filter(df["country"] == "ES")
```

Same `ref()`, same DAG, same tests. Requires Python-capable warehouse.

Session 5 — Practice: foundation

This session is hands-on, but the **concepts** likely on the exam:

Fan-out problem: `orders` has 1 row per order, `order_items` has N rows per order. Joining directly **duplicates the order value** N times. Fix: build `int_order_items_summary` with `group by order_id`, then 1:1 join.

Why `int_order_shipping` centralizes the definition of "Late". If business changes definition (e.g., "Late = > 2 days after estimate"), update in one place.

`mart_orders` **is a wide table** — analysts get one table with everything (status, items, shipping, payment). BI tools don't need to re-join.

`mart_revenue_by_segment` = `group by customer_segment`, `sum(total_amount)`, `avg(total_amount)`.
`mart_product_performance` = `group by product_id, category_name`, `sum(revenue)`, `sum(quantity)`.

Python model — `customers_enriched_python` uses Polars for messy text processing (email domains).

Session 6 — Materializations

Four core materializations:

Type	DDL	Pros	Cons	Default use
View	`CREATE VIEW`	Fast build, always fresh	Slow query	**Staging**
Table	`CREATE TABLE AS SELECT`	Fast query	Slow build, storage cost	**Marts**
Ephemeral	None — inlined as CTE	No clutter in DB	Hard to debug	Some intermediate
Incremental	Update existing table	Efficient on big data	Complex	Big fact tables (S10)

Performance trade-offs:

- Start with **views** (cheap)
- Move to **tables** when downstream query speed matters
- Use **ephemeral** sparingly (debug pain)

Session 6 (cont.) — Configuring materialization

Project-level (`dbt_project.yml`):

```
models:
  dbt_ie:
    staging:      {+materialized: view}
    intermediate: {+materialized: ephemeral}
    marts:        {+materialized: table}
```

Model-level (in the `.sql` file):

```
{{ config(materialized='table') }}
```

Model-level overrides project-level. Memorize this — common exam trap.

Grants in `dbt_project.yml`:

```
models:
  +grants:
    select: ['reporter', 'analyst']
```

dbt runs `GRANT` after creating the model.

Session 7 — Packages

Packages declared in `packages.yml`:

```yaml
packages:
  - package: dbt-labs/dbt_utils
    version: 1.1.1
  - package: dbt-labs/codegen
    version: 0.12.1
  - package: calogica/dbt_expectations
    version: 0.10.1
```

Install: `dbt deps`. Browse: **hub.getdbt.com**.

Key packages:

Package	Purpose
dbt_utils	Surrogate keys, unions, pivots, generic SQL helpers
codegen	Generate YAML/SQL boilerplate from sources
dbt_expectations	Great Expectations–style data quality tests

Session 7 (cont.) — Jinja basics

Syntax	Meaning
`{{ ... }}`	Expression — output a value
`{% ... %}`	Statement — control flow / set / for / if
`{# ... #}`	Comment (not output)
`{% set x = 10 %}`	Variable assignment
`{% if cond %} ... {% endif %}`	Conditional
`{% for item in list %} ... {% endfor %}`	Loop
`loop.first`, `loop.last`, `loop.index`	Loop state
`{{ target.name }}`	The current target ("dev", "prod"...)

`target.name` filter example (limit data in dev):

```
{% if target.name == 'dev' %}
  where order_date > '2024-01-01'
{% endif %}
```

Session 7 (cont.) — Macros & dbt-utils

Custom macros in `macros/`:

```
{% macro cents_to_dollars(column_name) %}
    (({{ column_name }} / 100)::numeric(16, 2)
{% endmacro %}
```

Use as `{{ cents_to_dollars('amount_cents') }}`.

dbt-utils essentials:

- `dbt_utils.generate_surrogate_key(['col1', 'col2'])` — hash-based composite PK
- `dbt_utils.union_relations(relations=[ref('a'), ref('b')])` — vertical concat

Git workflow (testable on cert):

1. `git checkout -b feature/foo`
2. Commit with meaningful messages
3. Push to remote
4. Open PR for review
5. Merge after approval — **atomic commits**: one logical change per commit

"If asked, answer" — quick recall

- **Materialization for staging?** `view`
- **Materialization for marts?** `table`
- **Where do tests live?** YAML next to model, or `tests/` for singular
- **What does `dbt build` do?** run + test + seed + snapshot in DAG order
- **`+model` means?** include upstream (ancestors)
- **`model+` means?** include downstream (descendants)
- **Where is compiled SQL?** `target/compiled/`
- **Where is executed SQL?** `target/run/`
- **`source()` vs `ref()`?** `source()` for raw tables in `sources.yml`. `ref()` for dbt-built models AND seeds
- **Where do macros live?** `macros/`
- **How to install packages?** `dbt deps` (reads `packages.yml`)
- **Test for duplicates?** `unique`
- **Test for NULLs?** `not_null`
- **Test for FK integrity?** `relationships`
- **Test for allowed values list?** `accepted_values`
- **Can seeds be `ref()`d?** Yes
- **Is `target/` versioned?** No — gitignored

Good luck

Then go take the 50-question quiz.

Practice questions → `practice_questions.html`
Quick reference → `cheatsheet.pdf`